

1. Navedite glavne arhitekturne značilnosti porazdeljenih sistemov.

- *vozlišča so povezana v omrežje*
- *vsako vozlišče ima ločen pomnilniški naslovni prostor*
- *komunikacija med vozlišči poteka z izmenjevanjem sporočil preko omrežja*

2. Podajte razliko med sočasnim in vzporednim izvajanjem procesov.

- *sočasno izvajanje: več niti v procesu se izvaja v istem časovnem oknu na enem procesorju; časovno okno je razdeljeno na več časovnih rezin v katerih se niti izvajajo zaporedno*
- *vzporedno izvajanje: različne niti se v istem trenutku izvajajo na različnih procesorskih jedrih*

3. V več sočasnih gorutinah `addOneByOne` povečujemo globalni števec `counter`. Vsaka gorutina ga mora povečati za ena, dokler ne doseže vrednosti `N`. Popravite spodnjo kodo, da bo skladna z zahtevami.

- *slabo napisano besedilo, zato upoštevane različne rešitve, če so le pravilno vključevale ključavnice*

```
var m sync.Mutex
var counter int = 0

func addOneByOne(N int) {
    for i:= 0; i < N; i++ {

        m.Lock()
        counter ++
        m.Unlock()

    }
}
```

4. V jeziku go želimo napisati program, v katerem glavna gorutina zažene `G` gorutin, ki sočasno izvajajo funkcijo `Workload`. Vsaka gorutina mora izvesti `N` obhodov zanke. Dopolnite spodnjo kodo tako, da bodo gorutine nadaljevale z naslednjim obhodom zanke ($i+1$) šele potem, ko bodo vse zaključile obdelavo v trenutnem obhodu zanke (i).

```
var g int = 0
var m = make(chan struct{}, 1)
var d = make(chan struct{}, 1)

func Workload(id int, G int, N int) {
    // zanka
    for i:= 0; i < N; i++ {

        // obdelava

        m <- struct{}{}
        g++
        if g == G {
            for i := 0; i < G; i++ {
                d <- struct{}{}
            }
        }
        <- m
        <- d
    }
}
```

```

        m <- struct{}{}
        g--
        if g == 0 {
            for i := 0; i < G; i++ {
                d <- struct{}{}
            }
        }
        <- m
        <- d
    }
}

```

5. V porazdeljenem sistemu imamo procese P1, P2 in P3. Vsak proces beleži interne dogodke in dogodke izmenjevanja sporočil. Dogodki na procesih si sledijo v naslednjem vrstnem redu: P1: *a, b, c*, P2: *d, e, f, g* in P3: *h, i, j*. Pari dogodkov (*e, b*), (*c, i*) in (*j, f*) označujejo izmenjevanje sporočil – prvi dogodek v paru označuje pošiljanje, drugi pa sprejemanje sporočila. Določite vrednost Lamportove ure ob dogodku *g*.

Lamportova ura ob posameznih dogodkih:

- *a: 1, b: 3, c: 4*
- *d: 1, e: 2, f: 7, **g: 8***
- *h: 1, i: 5, j: 6*

6. Na sistemu za veržno replikacijo želimo podpreti branje iz shrambe preko vseh vozlišč v verigi. Predlagajte rešitev, ki vključuje vpisovanje podatkov v shrambo in branje iz nje.

- *v fazi vpisovanja na vozlišču zapis označimo kot umazan dokler ni potrjen*
- *če pri branju zapis ni umazan, vozlišče vrne odjemalcu zahtevano vrednost*
- *če je zapis pri branju umazan, vozlišče vpraša rep za pravo vrednost in jo posreduje odjemalcu*

7. V gručo za replikacijo z voditeljem (algoritem raft) je vključenih pet procesov. Zaradi težav v omrežju gruča razpade na dva dela: v delu A sta voditelj in sledilec, v delu B pa trije sledilci. V delu B izvolijo novega voditelja in odjemalci dodajo nekaj vnosov v dnevnik. Natančno opišite kaj se zgodi potem, ko so težave v omrežju odpravljene?

- *voditelj B pošlje razširja dnevniške zapise (ali srčni utrip) s kasnejšim volilnim obdobjem*
- *voditelj iz A zato sestopi in postane sledilec*
- *voditelj iz B sledilcema iz A pošlje manjkajoče dnevniške zapise*

8. Primerjajte nestropno razširjanje sporočil in razširjanje sporočil z govoricami glede zanesljivosti in učinkovitosti.

- *pri nestrpnem pošiljanju vsak proces ob prvem prejemu sporočila tega prepošlje vsem ostalim procesom; časovna zahtevnost $O(n^2)$*
- *pri razširjanju z govoricami vsak proces ob prvem prejemu sporočila tega prepošlje p naključno izbranim procesom; časovna zahtevnost $O(n)$*

nestrpnost pošiljanje je bolj zanesljivo; za $p \geq 3$ je verjetnost, da sporočilo ne pride do vseh procesov precej majhna

9. Grafični pospeševalniki imajo zaradi hierarhične arhitekture procesorjev in pomnilnikov v programskem modelu hierarhično organizirane tudi niti. Preko katerega pomnilnika si niti najučinkoviteje izmenjujeta podatke, če se nahajata a) v istem bloku niti ali b) v različnih blokih niti?

- *niti v istem bloku preko skupnega pomnilnika v računski enoti*
- *niti v različnih blokih preko globalnega pomnilnika grafične kartice*

10. Imamo vektorja $\mathbf{a}$ in $\mathbf{b}$ dolžine n , ki jih želimo na grafičnem pospeševalniku sešteti v vektor $\mathbf{c} = \mathbf{a} + \mathbf{b}$. Popravite ščepec `vectorAdd` tako, da bo vsoto izračunal pravilno za vsako enodimenzionalno konfiguracijo mreže niti.

```
__global__ void vectorAdd(float *c, const float *a, const float *b, int n) {  
  
    int gid = threadIdx.x + blockDim.x * blockIdx.x;  
  
    while (gid < n) {  
        c[gid] = a[gid] + b[gid];  
        gid += blockDim.x * blockDim.x;  
    }  
}
```